

UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II



Network and Cloud Infrastructures Security Project Work

Anno Accademico 2024/2025

Professori:

Prof. Giorgio Ventre

Prof. Alessio Botta

Prof. Giovanni Stanco

Prof. Annalisa Navarro

Prof. Roberto Canonico

Studenti

Antonio Maria Improta M63001852

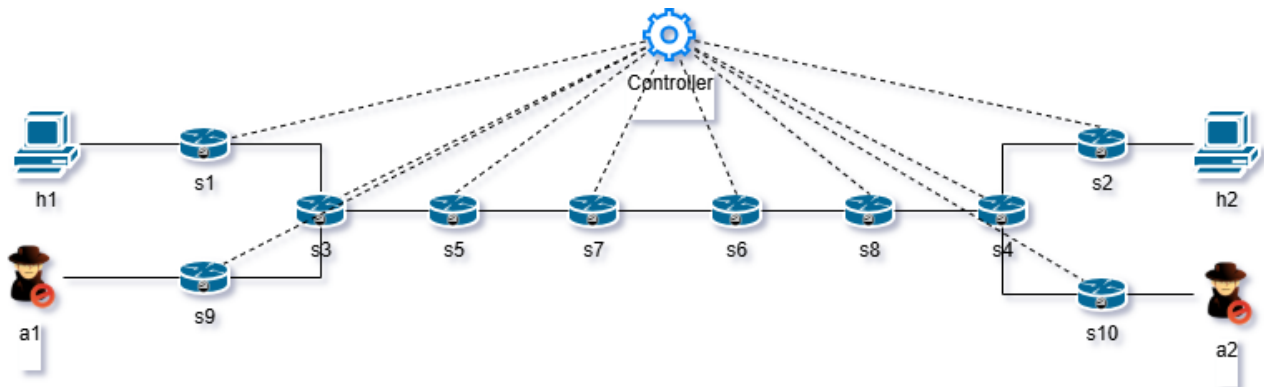
Mirko Fanini M63001731

1. Network Setup	3
2. Controller	4
2.1. MonitoringModule	6
2.2. PolicyModule	8
2.3. EnforcementModule	11
2.4. API REST	13
3. Results	15
3.1. Simulazione traffico lecito	15
3.2. Simulazione singolo attacco DoS	16
3.3. Simulazione doppio attacco DoS	18
3.4. Simulazione Threshold dinamica	22
4. Utilizzo API REST	25
4.1. Metodo GET Blocklist	25
4.2. Metodo POST	26
4.3. Metodo DELETE	27
4.4. Metodo GET Info	28

1. Network Setup

Il nostro progetto prevede l'implementazione dei seguenti punti richiesti:

- Over blocking
- Static Treshold
- Controller-Centric Blocking Decisions
- Lack of Modular Detection and Mitigation Design
- Topology Sensitivity



Abbiamo implementato una topologia priva di cicli composta da due host (h1,h2), due attaccanti (a1, a2) e 10 switch, come richiesto dal punto “Topology Sensitivity”. Si riporta di seguito il codice:

```
self.net = Mininet(controller=RemoteController, link=TCLink)

info("*** Starting controller\n")
c1 = self.net.addController('c1', controller=RemoteController)
c1.start()

info("*** Adding hosts\n")
self.h1 = self.net.addHost('h1', mac='00:00:00:00:00:01', ip='10.0.0.1/24')
self.h2 = self.net.addHost('h2', mac='00:00:00:00:00:02', ip='10.0.0.2/24')
self.a1 = self.net.addHost('a1', mac='00:00:00:00:00:A1', ip='10.0.0.101/24')
self.a2 = self.net.addHost('a2', mac='00:00:00:00:00:A2', ip='10.0.0.102/24')

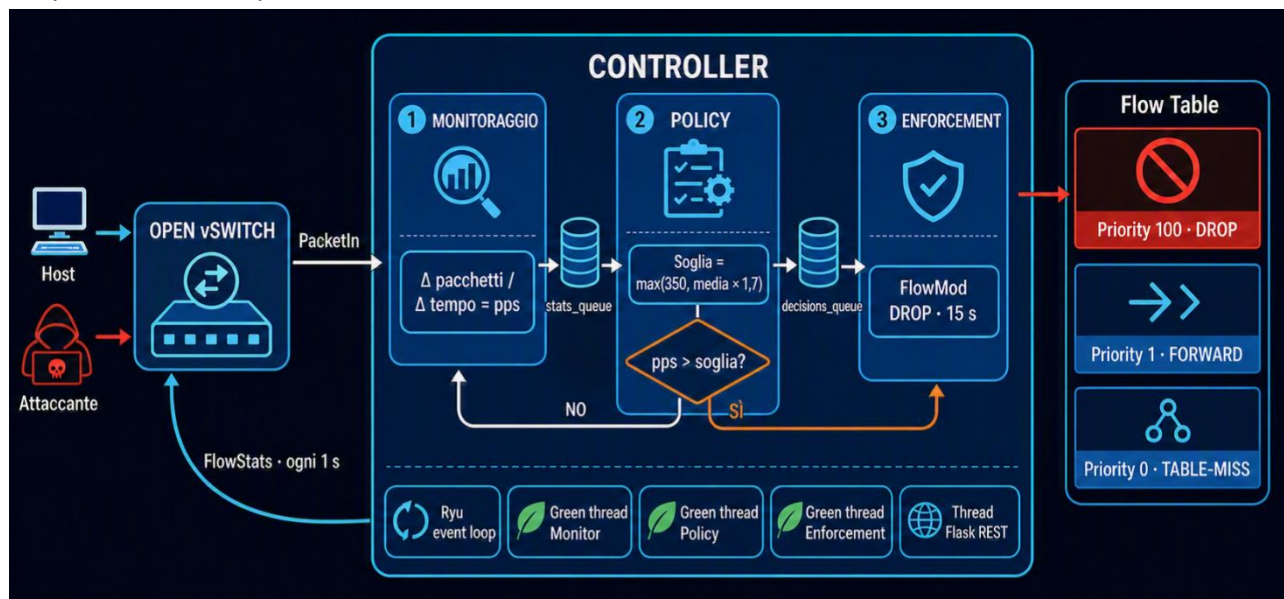
info("*** Adding 10 switches\n")
self.switches = {}
for idx in range(1, 11):
    name = f's{idx}'
    self.switches[name] = self.net.addSwitch(name, cls=OVSKernelSwitch)

info("*** Adding host access links\n")
self.net.addLink(self.h1, self.switches['s1'], bw=10, delay='0.5ms')
self.net.addLink(self.h2, self.switches['s2'], bw=10, delay='0.5ms')
self.net.addLink(self.a1, self.switches['s9'], bw=10, delay='0.5ms')
self.net.addLink(self.a2, self.switches['s10'], bw=10, delay='0.5ms')

info("*** Adding switch fabric links\n")
self.net.addLink(self.switches['s1'], self.switches['s3'], bw=6, delay='5ms')
self.net.addLink(self.switches['s9'], self.switches['s3'], bw=6, delay='5ms')
self.net.addLink(self.switches['s3'], self.switches['s5'], bw=7, delay='10ms')
self.net.addLink(self.switches['s5'], self.switches['s7'], bw=7, delay='10ms')
self.net.addLink(self.switches['s7'], self.switches['s6'], bw=7, delay='12ms')
self.net.addLink(self.switches['s6'], self.switches['s8'], bw=7, delay='10ms')
self.net.addLink(self.switches['s8'], self.switches['s4'], bw=7, delay='10ms')
self.net.addLink(self.switches['s4'], self.switches['s2'], bw=10, delay='5ms')
self.net.addLink(self.switches['s4'], self.switches['s10'], bw=6, delay='5ms')
```

2. Controller

La logica del Controller è stata organizzata in modo modulare per separare chiaramente le responsabilità, implementando così la feature richiesta di modularità.



L'architettura è composta da tre moduli principali, ognuno dei quali gira in un thread.

- Il MonitoringModule raccoglie le statistiche dagli switch e calcola gli indicatori utili.
- Il PolicyModule analizza questi dati, applica le regole di decisione e stabilisce se un flusso va bloccato.
- L'EnforcementModule infine esegue la decisione, installando le regole di drop e gestendo la blocklist.

Inoltre, al fine di implementare la possibilità per un admin esterno di interagire con il sistema, sono state implementate delle API REST, attraverso le quali è possibile interrogare la blocklist, aggiungere manualmente un flusso da bloccare, rimuovere una regola specifica oppure ricevere info riassuntive sulla blocklist (flussi totali bloccati, flussi expired ecc..).

Dal punto di vista architetturale, il controller funge da punto di ingresso dell'API REST, tramite un thread daemon in ascolto in background. Quindi le chiamate partono dal controller principale, che riceve la richiesta HTTP e la smista al modulo di enforcement. Questo contiene la logica effettiva di gestione della blocklist e dell'applicazione delle regole OpenFlow.

Riportiamo di seguito le componenti principali del codice del Controller.

Parametri istanziati dal controller:

```
OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
FLOW_WINDOW_SEC = 1.0
FLOW_PKT_THRESHOLD = 350
FLOW_BLOCK_SECONDS = 15
STATS_POLL_SEC = 1.0
ADAPTIVE_THRESHOLD_ENABLED = True
ADAPTIVE_HISTORY_SIZE = 12
ADAPTIVE_MULTIPLIER = 1.7
ADAPTIVE_MIN_SAMPLES = 3
WHITELIST_MACS = set()
WHITELIST_ETHERTYPES = {
    ether_types.ETH_TYPE_ARP,
    ether_types.ETH_TYPE_LLDP,
}
```

Come accennato in precedenza il Controller avvia i vari moduli e le API REST come thread:

```
self.monitoring_module.start()
self.policy_module.start()
self.enforcement_module.start()
self.api_thread = threading.Thread(target=self.start_rest_api, daemon=True)
self.api_thread.start()
```

La funzione *flow_key*, permette di rappresentare un flusso come tupla unica tramite le informazioni di dpid, porta, src, dest e eth type.

```
def flow_key(self, dpid, in_port, src, dst, eth_type):
    return (dpid, in_port, src, dst, eth_type)
```

La funzione *flow_stats_reply_handler* si occupa di rispondere alle richieste di OFPFlowStatsRequest, inviate dal modulo di monitoring, chiamando la funzione di tale modulo chiamata *process_flow_stats_reply*.

```
@set_ev_cls(ofp_event.EventOFPFlowStatsReply, MAIN_DISPATCHER)
def flow_stats_reply_handler(self, ev):
    msg = ev.msg
    datapath = msg.datapath
    self.monitoring_module.process_flow_stats_reply(datapath, msg.body)
```

La funzione `state_change_handler` gestisce il cambio di stato (connect/disconnect) degli switch e permette di registrare il datapath da utilizzare per lo scambio dei pacchetti.

```
@set_ev_cls(ofp_event.EventOFPSwitchChange, [MAIN_DISPATCHER, DEAD_DISPATCHER])
def state_change_handler(self, ev):
    datapath = ev.datapath
    if ev.state == MAIN_DISPATCHER:
        if datapath.id not in self.datapaths:
            self.logger.info(f"Register datapath dpid={datapath.id}")
            self.datapaths[datapath.id] = datapath
    elif ev.state == DEAD_DISPATCHER:
        if datapath.id in self.datapaths:
            self.logger.info(f"Unregister datapath dpid={datapath.id}")
            del self.datapaths[datapath.id]
```

2.1. MonitoringModule

Il MonitoringModule riceve la lista dei datapaths dal controller ed esegue il polling periodico dei FlowStats inviando richieste agli switch.

```
def monitor_loop(self):
    while True:
        try:
            # Ottiene la lista dei datapaths connessi dal controller
            datapaths = list(getattr(self.controller, 'datapaths', {}).values())

            # Chiede periodicamente stats agli switch
            for datapath in datapaths:
                self.request_flow_stats(datapath)
            hub.sleep(self.stats_poll_interval)

        except Exception as e:
            self.controller.logger.error(f"MonitoringModule error: {e}")
            hub.sleep(self.stats_poll_interval)

def request_flow_stats(self, datapath):
    # Invia OFPFlowStatsRequest agli switch
    try:
        parser = datapath.ofproto_parser
        req = parser.OFPFlowStatsRequest(datapath)
        datapath.send_msg(req)

    except Exception as e:
        self.controller.logger.error(f"Failed to request stats from dpid={datapath.id}: {e}")
```

Quando riceve il messaggio di risposta dagli switch, processa i FlowStats ricevuti, calcola il rate dei pacchetti relativo a ciascun flusso e consegna i dati al PolicyModule, tramite coda condivisa.

```
def process_flow_stats_reply(self, datapath, flow_stats_body):
    # flow_stats_body: Lista di stat objects dalla OpenFlowReply

    now_ts = time.time()
    dpid = datapath.id

    with self.flow_stats_lock:
        for stat in flow_stats_body:
            # Considera solo i flussi di forwarding (priority=1)
            if stat.priority != 1:
                continue

            in_port = stat.match.get('in_port')
            src = stat.match.get('eth_src')
            dst = stat.match.get('eth_dst')
            eth_type = stat.match.get('eth_type')

            # Se il match non contiene i campi attesi, salta
            if None in (in_port, src, dst, eth_type):
                continue

            flow_key = (dpid, in_port, src, dst, eth_type)
            prev = self.flow_stats_prev.get(flow_key)
            self.flow_stats_prev[flow_key] = {'ts': now_ts, 'pkt_count': stat.packet_count}

            if prev is None:
                continue

            delta_t = now_ts - prev['ts']
            if delta_t <= 0:
                continue

            delta_pkts = stat.packet_count - prev['pkt_count']
            # Counter reset (riavvio switch o flow ricreata)
            if delta_pkts < 0:
                continue

            pps = delta_pkts / delta_t

            # Crea un evento con tutti i dati rilevanti
            flow_event = {
                'flow_key': flow_key,
                'datapath': datapath,
                'in_port': in_port,
                'src': src,
                'dst': dst,
                'eth_type': eth_type,
                'pps': pps,
                'packet_count': stat.packet_count,
                'byte_count': stat.byte_count,
                'duration_sec': stat.duration_sec,
                'timestamp': now_ts,
            }

        # Consegna al Policy Module
        self.stats_queue.put(flow_event)
```

2.2. PolicyModule

Il PolicyModule riceve gli eventi dal MonitoringModule e applica ad ognuno la logica di decisione. Se la logica genera una decisione di blocco, questa viene passata al modulo di EnforcementModule tramite una coda condivisa.

```
def policy_loop(self):
    while True:
        try:
            # Ottiene il prossimo evento dal MonitoringModule
            try:
                flow_event = self.stats_queue.get(timeout=0.5)
            except Empty:
                hub.sleep(0.1)
                continue

            # Applica la logica di policy
            decision = self.evaluate_flow(flow_event)

            # Se la policy decide di bloccare, consegna la decisione
            if decision is not None:
                self.decisions_queue.put(decision)

        except Exception as e:
            self.controller.logger.error(f"PolicyModule error: {e}")
            hub.sleep(0.1)
```

Inizialmente, calcola la threshold dinamica utilizzando lo storico recente, in particolare se il numero di sample registrati è inferiore a ADAPTIVE_MIN_SAMPLES, viene utilizzata la threshold statica, in quanto la media non sarebbe ancora significativa.

```
def get_adaptive_threshold_pps(self, flow_key, src=None, dst=None):
    # Threshold statica
    static_floor_pps = self.flow_pkt_threshold / self.flow_window_sec

    if not self.adaptive_threshold_enabled:
        return static_floor_pps

    with self.flow_pps_history_lock:
        history = list(self.flow_pps_history.get(flow_key, ()))

    if len(history) < self.adaptive_min_samples:
        return static_floor_pps

    # Threshold dinamica
    baseline_pps = mean(history)
    threshold_pps = baseline_pps * self.adaptive_multiplier

    return max(static_floor_pps, threshold_pps)

def record_flow_sample(self, flow_key, pps):
    with self.flow_pps_history_lock:
        self.flow_pps_history[flow_key].append(pps)
```


Poi, verifica se si tratta di un flusso in whitelist oppure di un flusso bloccato di recente, al fine di evitare regole duplicate in un intervallo molto breve (1 secondo).

Tale controllo avviene grazie alle funzioni *is_whitelisted* e *is_recently_blocked*.

```
self.recently_blocked = {} # flow_key -> timestamp
self.recently_blocked_lock = threading.Lock()
```

```
def is_whitelisted(self, src, dst, eth_type):
    return (
        eth_type in self.whitelist_ethertypes
        or src in self.whitelist_macs
        or dst in self.whitelist_macs
    )

def is_recently_blocked(self, flow_key, now_ts):
    # Verifica se il flusso è stato bloccato di recente
    with self.recently_blocked_lock:
        last_blocked_ts = self.recently_blocked.get(flow_key)
        if last_blocked_ts is None:
            return False

        # Se è passato meno di 1 secondo, considera bloccato di recente
        if now_ts - last_blocked_ts < 1.0:
            return True

        del self.recently_blocked[flow_key]
        return False

def mark_as_blocked_recently(self, flow_key, now_ts):
    with self.recently_blocked_lock:
        self.recently_blocked[flow_key] = now_ts
```

Infine, verifica se il rate dei pacchetti supera la threshold e genera la decisione di blocco.

```
def evaluate_flow(self, flow_event):
    # Applica la logica di decisione

    flow_key = flow_event['flow_key']
    src = flow_event['src']
    dst = flow_event['dst']
    eth_type = flow_event['eth_type']
    pps = flow_event['pps']
    now_ts = flow_event['timestamp']

    # Calcola la threshold dinamica
    threshold_pps = self.get_adaptive_threshold_pps(flow_key)

    # Verifica la whitelist
    if self.is_whitelisted(src, dst, eth_type):
        self.record_flow_sample(flow_key, pps)
        return None

    # Verifica se si tratta di un flusso recentemente bloccato
    if self.is_recently_blocked(flow_key, now_ts):
        self.record_flow_sample(flow_key, pps)
        return None

    # Verifica se supera la threshold
    if pps <= threshold_pps:
        self.record_flow_sample(flow_key, pps)
        return None

    # Genera la decisione di blocco
    decision = {
        'action': 'block',
        'flow_key': flow_key,
        'datapath': flow_event['datapath'],
        'in_port': flow_event['in_port'],
        'src': src,
        'dst': dst,
        'eth_type': eth_type,
        'pps': pps,
        'threshold_pps': threshold_pps,
        'block_duration_sec': self.flow_block_seconds,
        'timestamp': now_ts,
    }

    # Marca il flusso come bloccato di recente
    self.mark_as_blocked_recently(flow_key, now_ts)
    self.record_flow_sample(flow_key, pps)

    self.controller.logger.info(
        f"PolicyModule DECISION: Block flow {flow_key} "
        f"(pps={pps:.2f} > adaptive_threshold={threshold_pps:.2f})"
    )

    return decision
```

2.3. EnforcementModule

Lo EnforcementModule riceve le decisioni dal PolicyModule. Se la decisione è una decisione di blocco, viene invocata la funzione *enforce_block*.

```
def enforcement_loop(self):
    while True:
        try:
            # Ottiene la prossima decisione
            try:
                decision = self.decisions_queue.get(timeout=0.5)
            except Empty:
                hub.sleep(0.1)
                continue

            # Applica la decisione
            if decision['action'] == 'block':
                self.enforce_block(decision)

        except Exception as e:
            self.controller.logger.error(f"EnforcementModule error: {e}")
            hub.sleep(0.1)
```

La funzione *enforce_block* legge la decisione, aggiunge il flusso alla blocklist condivisa e installa la regola di blocco sullo switch.

```
def enforce_block(self, decision):
    flow_key = decision['flow_key']
    datapath = decision['datapath']
    in_port = decision['in_port']
    src = decision['src']
    dst = decision['dst']
    eth_type = decision['eth_type']
    block_duration_sec = decision['block_duration_sec']
    pps = decision['pps']
    now_ts = decision['timestamp']

    # Aggiunge alla blocklist condivisa
    with self.blocklist_lock:
        expire_ts = now_ts + block_duration_sec
        self.shared_blocklist[flow_key] = expire_ts

    try:
        parser = datapath.ofproto_parser
        drop_match = parser.OFPMatch(
            in_port=in_port,
            eth_src=src,
            eth_dst=dst,
            eth_type=eth_type,
        )
        self.install_drop_flow(datapath, drop_match, block_duration_sec)

        self.controller.logger.warning(
            f"EnforcementModule: BLOCKED flow {flow_key} "
            f"for {block_duration_sec}s (pps={pps:.2f}) "
            f"dpid={datapath.id} in_port={in_port} "
            f"src={src} dst={dst} eth_type={eth_type}"
        )

    except Exception as e:
        self.controller.logger.error(
            f"EnforcementModule: Failed to install drop rule for {flow_key}: {e}"
        )

def install_drop_flow(self, datapath, match, hard_timeout):
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    mod = parser.OFPFlowMod(
        datapath=datapath,
        priority=100,
        match=match,
        instructions=[], # Nessuna azione = drop
        hard_timeout=hard_timeout,
    )
    datapath.send_msg(mod)
```

Inoltre, l'EnforcementModule implementa funzioni di gestione della blocklist condivisa. Di seguito si riportano le funzioni per verificare se un flusso è bloccato, per restituire la blocklist attuale e per aggiungere/rimuovere un flusso dalla blocklist.

C'è da notare che un flusso expired rimane nella blocklist fino a quando non viene effettuato un nuovo controllo su di esso tramite la funzione *is_flow_blocked*, richiamata in seguito a un nuovo evento di *PacketIn*. Questa scelta ci ha permesso, attraverso le API REST, di ottenere statistiche utili sia sui flussi bloccati attivi sia su quelli expired.

Una possibile evoluzione futura sarebbe l'introduzione di un thread dedicato alla pulizia, in grado di rimuovere quasi immediatamente dalla blocklist i flussi scaduti.

```
def is_flow_blocked(self, flow_key, now_ts=None):
    # Ritorna True se il flusso è bloccato e non scaduto
    if now_ts is None:
        now_ts = time.time()
    with self.blocklist_lock:
        expire_ts = self.shared_blocklist.get(flow_key)
        if expire_ts is None or now_ts >= expire_ts:
            self.shared_blocklist.pop(flow_key, None)
            return False
    return True

def add_to_blocklist(self, flow_key, block_duration_sec=None):
    if block_duration_sec is None:
        block_duration_sec = self.policy_module.flow_block_seconds
    with self.blocklist_lock:
        expire_ts = time.time() + block_duration_sec
        self.shared_blocklist[flow_key] = expire_ts
    return {'status': 'added', 'expire_ts': expire_ts}

def remove_from_blocklist(self, flow_key):
    with self.blocklist_lock:
        removed = self.shared_blocklist.pop(flow_key, None) is not None
    return {'status': 'removed' if removed else 'not_found'}

def get_blocklist(self):
    with self.blocklist_lock:
        return dict(self.shared_blocklist)
```

2.4. API REST

Riportiamo di seguito le API REST implementate.

Blocklist_info restituisce le statistiche della blocklist:

```
@app.route('/api/blocklist/info', methods=['GET'])
def blocklist_info():
    # Statistiche sulla blocklist
    blocklist = self.enforcement_module.get_blocklist()
    now = time.time()

    expired_count = sum(1 for ts in blocklist.values() if ts <= now)
    active_count = len(blocklist) - expired_count

    return jsonify({
        'status': 'success',
        'total_blocked_flows': len(blocklist),
        'active_blocked_flows': active_count,
        'expired_blocked_flows': expired_count,
        'default_block_duration_sec': self.FLOW_BLOCK_SECONDS,
        'timestamp': now
    }), 200
```

Remove_from_blocklist rimuove un flow dalla blocklist:

```
@app.route('/api/blocklist/<path:flow_key>', methods=['DELETE'])
def remove_from_blocklist(flow_key):
    # Rimuove un flow dalla blocklist
    try:
        flow_key_tuple = literal_eval(flow_key)
        result = self.enforcement_module.remove_from_blocklist(flow_key_tuple)
        status_code = 200 if result['status'] == 'removed' else 404
        return jsonify({
            'status': 'success',
            'message': f"Flow {result['status']}",
            'flow_key': flow_key
        }), status_code
    except Exception as e:
        return jsonify({'error': f'Invalid flow_key: {str(e)}'}), 400
```

Add_to_blocklist aggiunge un flow alla blocklist:

```
@app.route('/api/blocklist', methods=['POST'])
def add_to_blocklist():
    # Aggiunge un flow alla blocklist
    try:
        data = request.get_json(silent=True)
        if not data or 'flow_key' not in data:
            return jsonify({'error': 'Missing flow_key'}), 400

        flow_key = tuple(data['flow_key'])
        duration = data.get('duration', SimpleSwitch13.FLOW_BLOCK_SECONDS)

        if not isinstance(duration, (int, float)) or duration <= 0:
            return jsonify({'error': 'duration must be positive'}), 400

        result = self.enforcement_module.add_to_blocklist(flow_key, duration)

        return jsonify({
            'status': 'success',
            'message': 'Flow added to blocklist',
            'flow_key': str(flow_key),
            'block_duration_sec': duration,
            'expire_ts': result['expire_ts']
        }), 201

    except Exception as e:
        return jsonify({'error': str(e)}), 400
```

Get_blocklist permette di visualizzare i flussi bloccati:

```
@app.route('/api/blocklist', methods=['GET'])
def get_blocklist():
    # Visualizza tutti i flow attualmente bloccati
    blocklist = self.enforcement_module.get_blocklist()
    return jsonify({
        'status': 'success',
        'blocklist': serialize_blocklist(blocklist),
        'count': len(blocklist),
        'timestamp': time.time()
    }), 200
```

Serialize_blocklist permette di trasformare le informazioni riguardanti i flussi in formato JSON-friendly, al fine di essere inviate tramite http.

```
def serialize_blocklist(blocklist):
    #Converte la blocklist con timestamp in formato JSON-friendly
    now = time.time()
    return {
        str(key): {
            'expire_ts': expire_ts,
            'seconds_remaining': max(0, expire_ts - now),
            'expired': expire_ts <= now
        }
        for key, expire_ts in blocklist.items()
    }
```

3. Results

3.1. Simulazione traffico lecito

Abbiamo simulato un flusso TCP tra l'host h1 e l'host h2 e un flusso UDP lecito tra l'attaccante a1 e l'host h2, tramite i seguenti comandi:

Host h2 in ascolto sulle porte 5001 e 5002:

- h2 iperf -s -p 5001 -u -i 1 > H2_udp
- h2 iperf -s -p 5002 -i 1 > H2_tcp

Flussi TCP e UDP:

- h1 iperf -c 10.0.0.2 -b 3M -p 5002 -t 60
- a1 iperf -c 10.0.0.2 -u -b 2M -p 5001 -t 60

Ottenendo i seguenti risultati:

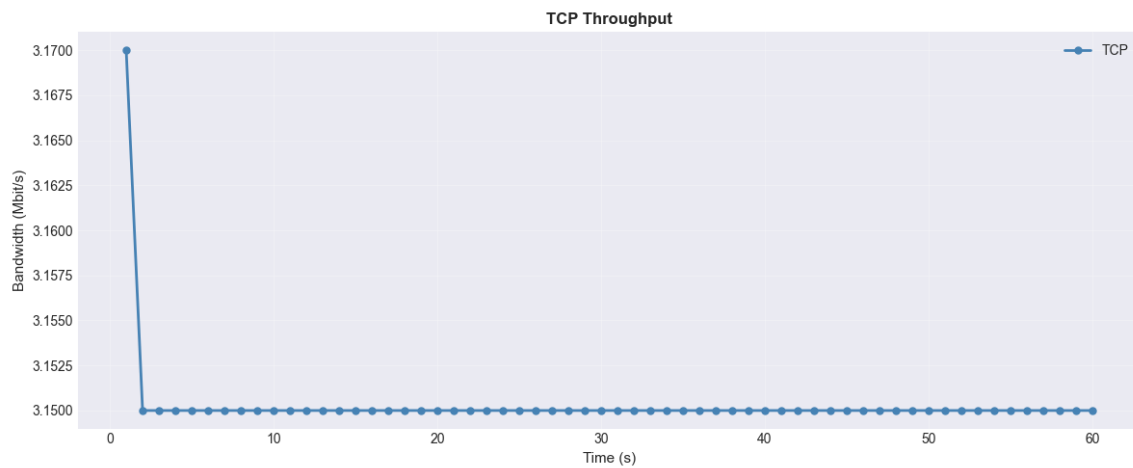


Figure 1 Traffico TCP da h1 a h2

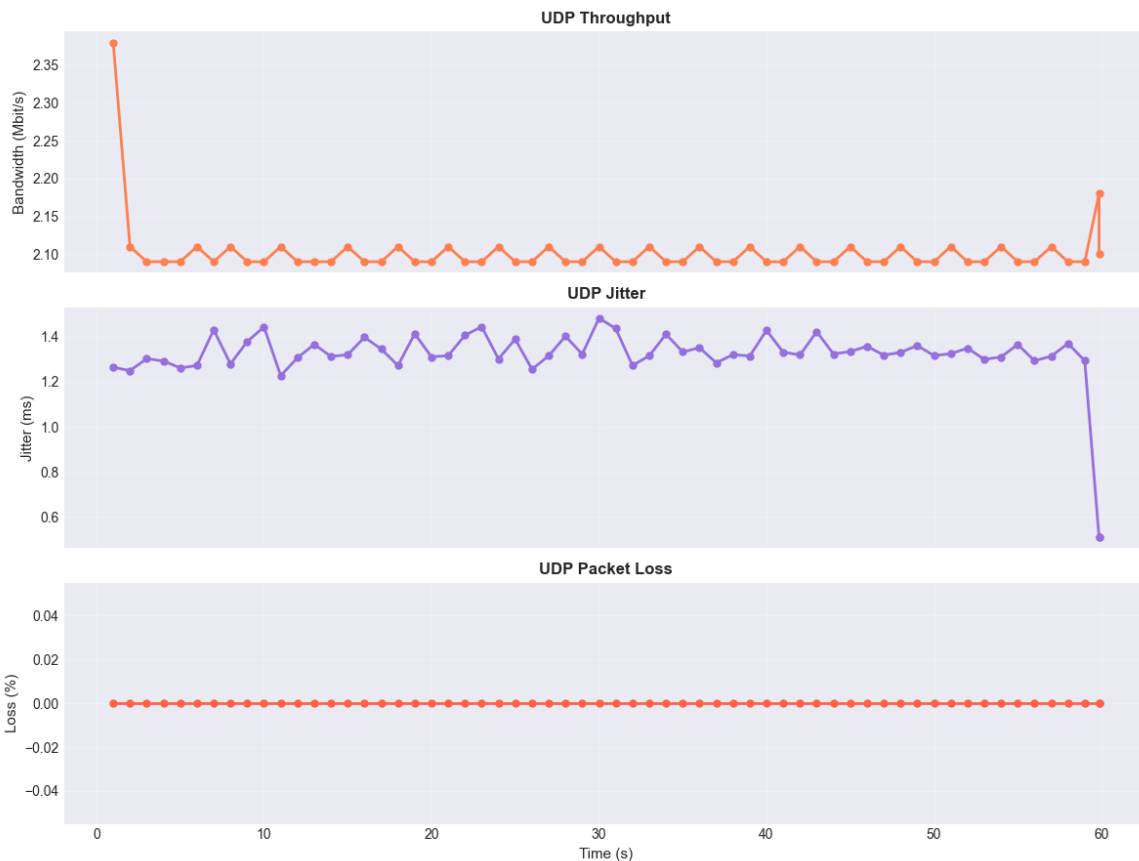


Figure 2 Traffico UDP da a1 a h2

Si evince che la bandwidth in entrambi i casi è stabile.

3.2. Simulazione singolo attacco DoS

Per simulare uno scenario di congestione critica, il bitrate dell'attaccante è stato elevato a 9 Mbps. Tale valore, sommato ai 3 Mbps generati dall'host h1, eccede la capacità nominale dei link di rete, causando, in assenza di meccanismi di controllo, una significativa perdita di pacchetti.

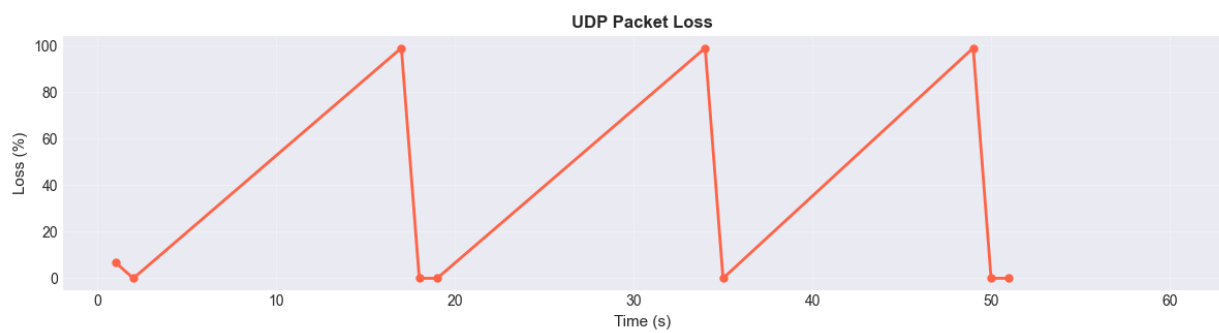
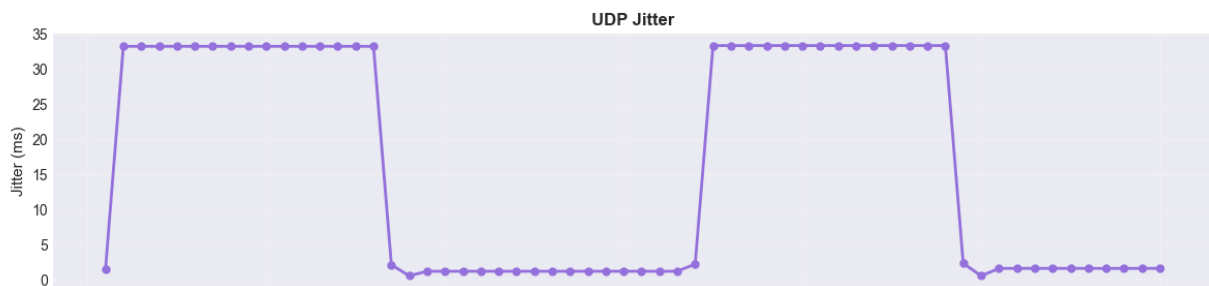
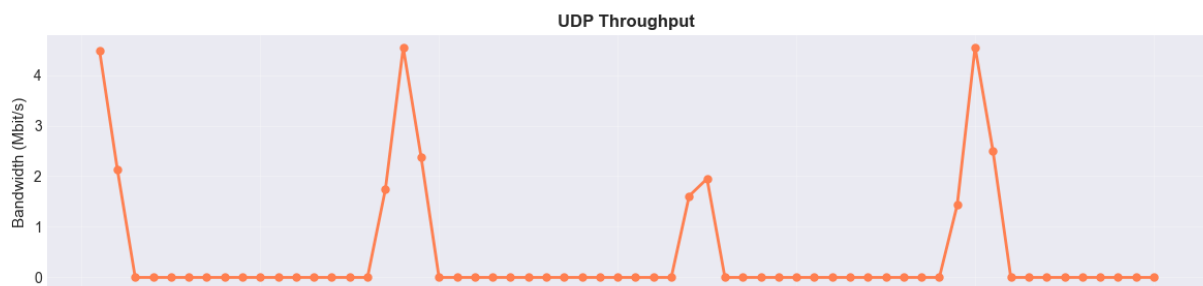
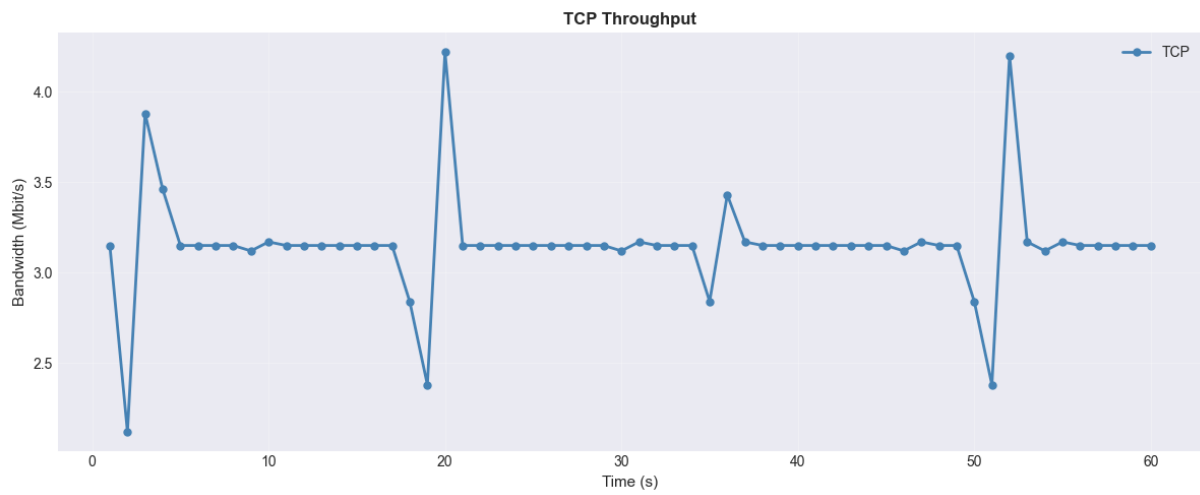
Comandi utilizzati:

Host h2 in ascolto sulle porte 5001 e 5002:

- h2 iperf -s -p 5001 -u -i 1 > H2_udp
- h2 iperf -s -p 5002 -i 1 > H2_tcp

Flussi TCP e UDP:

- h1 iperf -c 10.0.0.2 -b 3M -p 5002 -t 60
- a1 iperf -c 10.0.0.2 -u -b 9M -p 5001 -t 60



Notiamo come solamente il flusso UDP viene bloccato nel momento in cui eccede la threshold.

A dimostrazione del funzionamento del controller, riportiamo uno screen dell'output del Controller:

```
PolicyModule DECISION: Block flow (5, 1, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048) (pps=359.30 > adaptive_threshold=350.00)
PolicyModule DECISION: Block flow (3, 2, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048) (pps=487.04 > adaptive_threshold=350.00)
PolicyModule DECISION: Block flow (7, 1, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048) (pps=362.33 > adaptive_threshold=350.00)
PolicyModule DECISION: Block flow (8, 1, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048) (pps=369.28 > adaptive_threshold=350.00)
PolicyModule DECISION: Block flow (6, 1, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048) (pps=363.16 > adaptive_threshold=350.00)
PolicyModule DECISION: Block flow (4, 1, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048) (pps=370.26 > adaptive_threshold=350.00)
PolicyModule DECISION: Block flow (2, 2, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048) (pps=372.22 > adaptive_threshold=350.00)
EnforcementModule: BLOCKED flow (5, 1, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048) for 15s (pps=359.30) dpid=5 in_port=1 src=00:00:00:00:00:01 dst=00:00:00:00:00:02 eth_type=2048
```

Notiamo come viene bloccato il flusso con sorgente a1 (00:00:00:00:00:a1) e destinazione h2 (00:00:00:00:00:02).

Inoltre, grazie al comando:

- `sudo ovs-ofctl dump-flows s5`

Siamo in grado di visualizzare le regole installate nello switch s5, notiamo la penultima regola con priorità 100 con sorgente a1 e destinazione h2

```
cookie=0x0, duration=136.490s, table=0, n_packets=2, n_bytes=84, priority=1,arp,in_port="s5-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s5-eth1"
cookie=0x0, duration=136.445s, table=0, n_packets=12209, n_bytes=35545042, priority=1,ip,in_port="s5-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s5-eth2"
cookie=0x0, duration=136.317s, table=0, n_packets=12188, n_bytes=804472, priority=1,ip,in_port="s5-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s5-eth1"
cookie=0x0, duration=130.990s, table=0, n_packets=1, n_bytes=42, priority=1,arp,in_port="s5-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s5-eth2"
cookie=0x0, duration=119.749s, table=0, n_packets=3, n_bytes=126, priority=1,arp,in_port="s5-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:a1 actions=output:"s5-eth1"
cookie=0x0, duration=119.704s, table=0, n_packets=3383, n_bytes=5115096, priority=1,ip,in_port="s5-eth1",dl_src=00:00:00:00:00:a1,dl_dst=00:00:00:00:00:02 actions=output:"s5-eth2"
cookie=0x0, duration=86.404s, table=0, n_packets=2, n_bytes=84, priority=1,arp,in_port="s5-eth1",dl_src=00:00:00:00:00:a1,dl_dst=00:00:00:00:00:02 actions=output:"s5-eth2"
cookie=0x0, duration=0.593s, table=0, n_packets=373, n_bytes=563976, hard_timeout=15, priority=100,ip,in_port="s5-eth1",dl_src=00:00:00:00:00:a1,dl_dst=00:00:00:00:00:02 actions=drop
cookie=0x0, duration=197.877s, table=0, n_packets=287, n_bytes=34848, priority=0 actions=CONTROLLER:65535
```

3.3. Simulazione doppio attacco DoS

In questo paragrafo abbiamo simulato un doppio attacco Dos a partire da entrambi gli attaccanti. In particolare, abbiamo le seguenti comunicazioni:

- h1 -> h2: h1 iperf -c 10.0.0.2 -b 2M -p 5002 -t 100 &
- h2 -> h1: h2 iperf -c 10.0.0.1 -b 2M -p 5002 -t 100 &
- a1 -> h2: a1 iperf -c 10.0.0.2 -u -b 6M -p 5001 -t 100 &
- a2 -> h1: a2 iperf -c 10.0.0.1 -u -b 6M -p 5001 -t 100

Sia h1 che h2 si mettono in ascolto a priori sulle porte 5001 e 5002:

- h2 UDP h2 iperf -s -p 5001 -u -i 1 > H2_udp &
- h2 TCP: h2 iperf -s -p 5002 -i 1 > H2_tcp &
- h1 UDP: h1 iperf -s -p 5001 -u -i 1 > H1_udp &
- h1 TCP: h1 iperf -s -p 5002 -i 1 > H1_tcp &

Di seguito riportiamo i risultati:

Statistiche registrate sull'host H2:

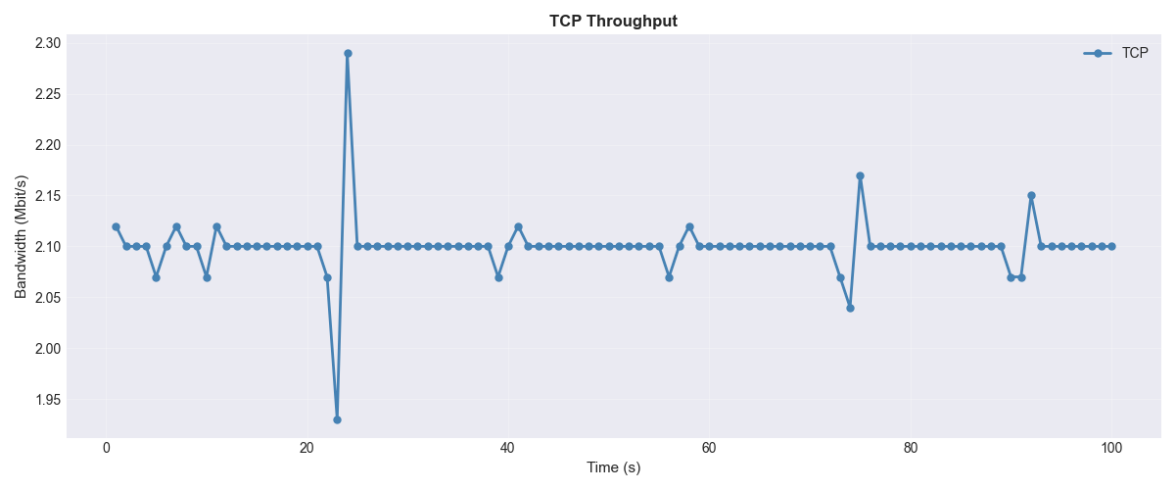


Figure 3 Traffico TCP da h1 a h2

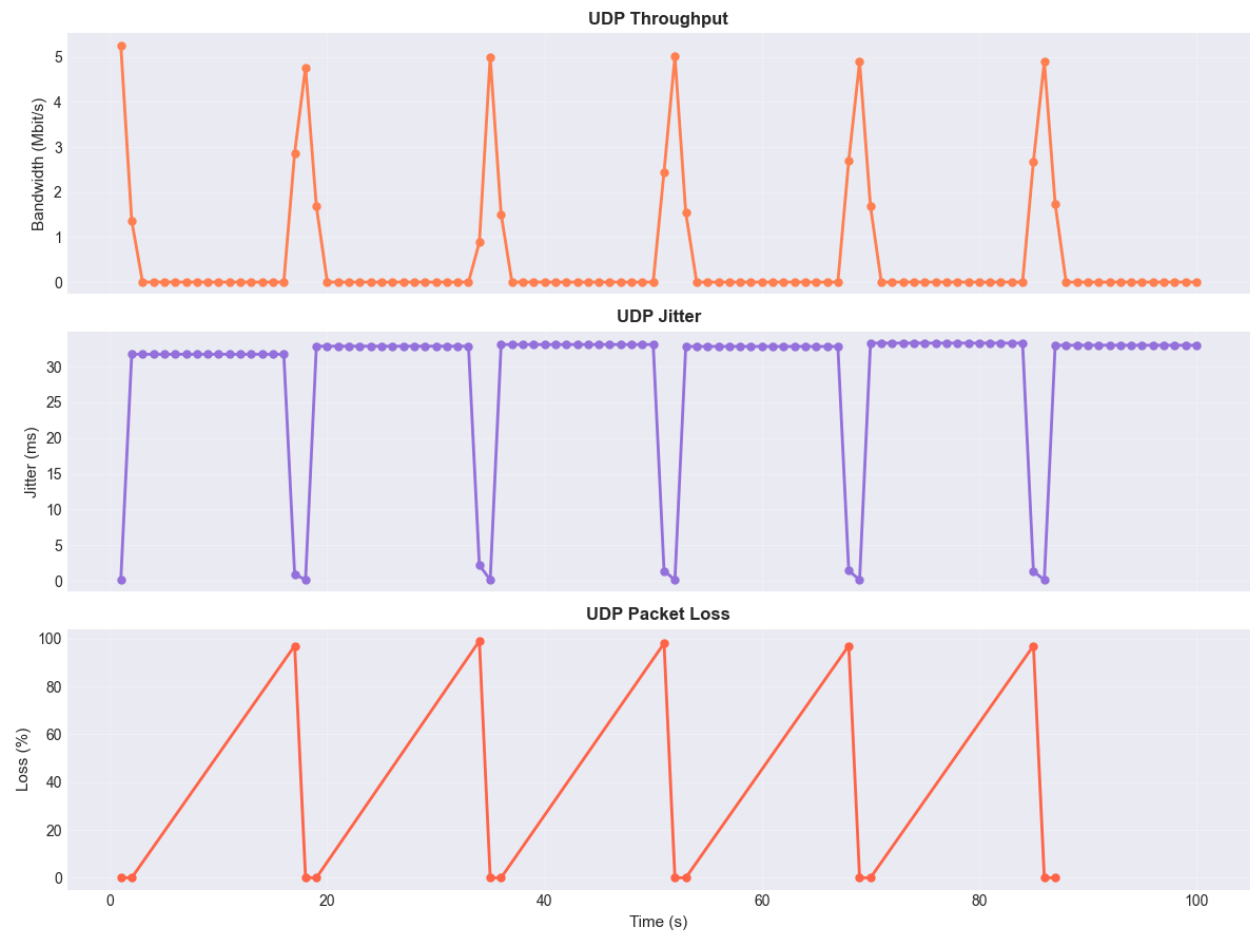


Figure 4 Attacco DoS UDP da a1 a h2

Statistiche registrate sull'host H1:

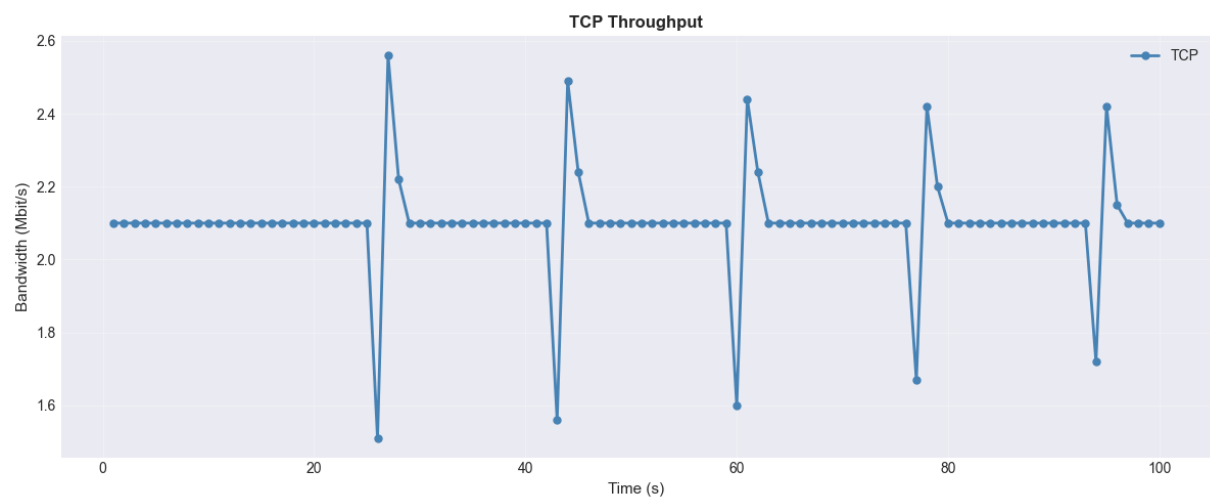


Figure 5 Flusso TCP da h2 a h1

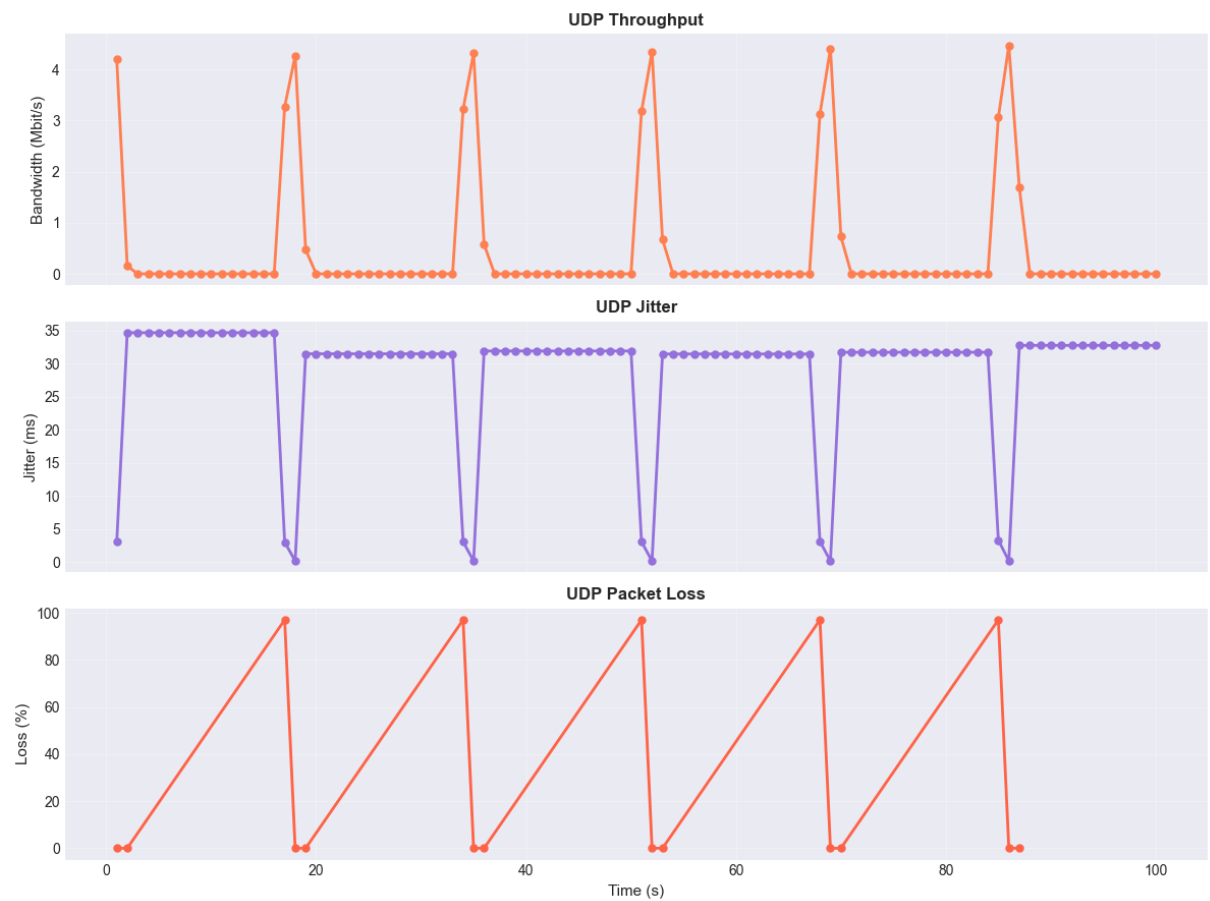


Figure 6 Attacco DoS UDP da a2 a h1

Anche qui riportiamo uno screen dell'output del controller:

```
PolicyModule DECISION: Block flow (5, 2, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048) (pps=440.67 > adaptive_threshold=350.00)
PolicyModule DECISION: Block flow (8, 2, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048) (pps=426.35 > adaptive_threshold=350.00)
PolicyModule DECISION: Block flow (4, 3, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048) (pps=494.34 > adaptive_threshold=350.00)
PolicyModule DECISION: Block flow (7, 2, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048) (pps=436.54 > adaptive_threshold=350.00)
PolicyModule DECISION: Block flow (10, 1, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048) (pps=533.06 > adaptive_threshold=350.00)
EnforcementModule: BLOCKED flow (5, 2, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048) for 15s (pps=440.67) dpid=5 in_port=2 src=00:00:00:00:00:00:a2 dst=00:00:00:00:00:00:01 eth_type=2048
EnforcementModule: BLOCKED flow (8, 2, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048) for 15s (pps=426.35) dpid=8 in_port=2 src=00:00:00:00:00:00:a2 dst=00:00:00:00:00:00:01 eth_type=2048
EnforcementModule: BLOCKED flow (4, 3, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048) for 15s (pps=494.34) dpid=4 in_port=3 src=00:00:00:00:00:00:a2 dst=00:00:00:00:00:00:01 eth_type=2048
EnforcementModule: BLOCKED flow (7, 2, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048) for 15s (pps=436.54) dpid=7 in_port=2 src=00:00:00:00:00:00:a2 dst=00:00:00:00:00:00:01 eth_type=2048
EnforcementModule: BLOCKED flow (10, 1, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048) for 15s (pps=533.06) dpid=10 in_port=1 src=00:00:00:00:00:00:a2 dst=00:00:00:00:00:00:01 eth_type=2048
```

Riportiamo inoltre le regole installate in alcuni degli switch, in particolare s5, s7, s8:

```
cookie=0x0, duration=64.382s, table=0, n_packets=0, n_bytes=0, priority=1,arp,in_port="s7-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s7-eth1"
cookie=0x0, duration=64.311s, table=0, n_packets=11717, n_bytes=17550634, priority=1,ip,in_port="s7-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s7-eth2"
cookie=0x0, duration=64.205s, table=0, n_packets=11740, n_bytes=17421080, priority=1,ip,in_port="s7-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s7-eth1"
cookie=0x0, duration=58.514s, table=0, n_packets=1, n_bytes=42, priority=1,arp,in_port="s7-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:a1 actions=output:"s7-eth1"
cookie=0x0, duration=58.443s, table=0, n_packets=2722, n_bytes=4115664, priority=1,ip,in_port="s7-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s7-eth2"
1 cookie=0x0, duration=53.705s, table=0, n_packets=2, n_bytes=84, priority=1,arp,in_port="s7-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:a2 actions=output:"s7-eth2"
1 cookie=0x0, duration=53.608s, table=0, n_packets=2877, n_bytes=4350024, priority=1,ip,in_port="s7-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s7-eth1"
1 a2,dl_dst=00:00:00:00:00:01 actions=output:"s7-eth1"
1 cookie=0x0, duration=32.393s, table=0, n_packets=1, n_bytes=42, priority=1,arp,in_port="s7-eth2",dl_src=00:00:00:00:00:a2,dl_dst=00:00:00:00:00:01 actions=output:"s7-eth1"
1 st=00:00:00:00:00:01 actions=output:"s7-eth1"
1 cookie=0x0, duration=24.074s, table=0, n_packets=0, n_bytes=0, priority=1,arp,in_port="s7-eth1",dl_src=00:00:00:00:00:a1,dl_dst=00:00:00:00:00:02 actions=output:"s7-eth2"
t=00:00:00:00:00:02 actions=output:"s7-eth2"
cookie=0x0, duration=6.149s, table=0, n_packets=5, n_bytes=7560, hard_timeout=15, priority=100,ip,in_port="s7-eth1",dl_src=00:00:00:00:00:a1,dl_dst=00:00:00:00:00:02 actions=drop
cookie=0x0, duration=1.157s, table=0, n_packets=218, n_bytes=329616, hard_timeout=15, priority=100,ip,in_port="s7-eth2",dl_src=00:00:00:00:00:a2,dl_dst=00:00:00:00:00:01 actions=drop
cookie=0x0, duration=390.574s, table=0, n_packets=220, n_bytes=27963, priority=0 actions=CONTROLLER:65535
```

Figure 7 Regole installate in s7

```
cookie=0x0, duration=72.259s, table=0, n_packets=0, n_bytes=0, priority=1,arp,in_port="s8-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s8-eth1"
cookie=0x0, duration=72.132s, table=0, n_packets=13093, n_bytes=19607530, priority=1,ip,in_port="s8-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s8-eth2"
cookie=0x0, duration=72.079s, table=0, n_packets=13120, n_bytes=19478240, priority=1,ip,in_port="s8-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s8-eth1"
cookie=0x0, duration=66.387s, table=0, n_packets=1, n_bytes=42, priority=1,arp,in_port="s8-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:a1 actions=output:"s8-eth1"
cookie=0x0, duration=66.263s, table=0, n_packets=2698, n_bytes=4079376, priority=1,ip,in_port="s8-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s8-eth2"
a1,dl_dst=00:00:00:00:00:02 actions=output:"s8-eth2"
cookie=0x0, duration=61.528s, table=0, n_packets=2, n_bytes=84, priority=1,arp,in_port="s8-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:a2 actions=output:"s8-eth2"
st=00:00:00:00:00:a2 actions=output:"s8-eth2"
cookie=0x0, duration=61.482s, table=0, n_packets=2713, n_bytes=4102056, priority=1,ip,in_port="s8-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s8-eth1"
a2,dl_dst=00:00:00:00:00:01 actions=output:"s8-eth1"
cookie=0x0, duration=40.270s, table=0, n_packets=1, n_bytes=42, priority=1,arp,in_port="s8-eth2",dl_src=00:00:00:00:00:a2,dl_dst=00:00:00:00:00:01 actions=output:"s8-eth1"
st=00:00:00:00:00:01 actions=output:"s8-eth1"
cookie=0x0, duration=31.892s, table=0, n_packets=0, n_bytes=0, priority=1,arp,in_port="s8-eth1",dl_src=00:00:00:00:00:a1,dl_dst=00:00:00:00:00:02 actions=output:"s8-eth2"
t=00:00:00:00:00:02 actions=output:"s8-eth2"
cookie=0x0, duration=13.997s, table=0, n_packets=5, n_bytes=7560, hard_timeout=15, priority=100,ip,in_port="s8-eth1",dl_src=00:00:00:00:00:a1,dl_dst=00:00:00:00:00:02 actions=drop
:00:00:00:00:a1,dl_dst=00:00:00:00:00:02 actions=drop
cookie=0x0, duration=9.005s, table=0, n_packets=356, n_bytes=538272, hard_timeout=15, priority=100,ip,in_port="s8-eth2",dl_src=00:00:00:00:00:a2,dl_dst=00:00:00:00:00:01 actions=drop
=00:00:00:00:00:a2,dl_dst=00:00:00:00:00:01 actions=drop
cookie=0x0, duration=398.353s, table=0, n_packets=233, n_bytes=29908, priority=0 actions=CONTROLLER:65535
```

Figure 8 Regole installate in s8


```

cookie=0x0, duration=75.284s, table=0, n_packets=0, n_bytes=0, priority=1,arp,in_port="s5-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s5-eth1"
cookie=0x0, duration=75.239s, table=0, n_packets=13689, n_bytes=20564370, priority=1,ip,in_port="s5-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s5-eth2"
cookie=0x0, duration=75.106s, table=0, n_packets=13751, n_bytes=20437390, priority=1,ip,in_port="s5-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s5-eth1"
cookie=0x0, duration=69.417s, table=0, n_packets=2, n_bytes=84, priority=1,arp,in_port="s5-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:a1 actions=output:"s5-eth1"
cookie=0x0, duration=69.368s, table=0, n_packets=3490, n_bytes=5276880, priority=1,ip,in_port="s5-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02 actions=output:"s5-eth2"
cookie=0x0, duration=64.632s, table=0, n_packets=2, n_bytes=84, priority=1,arp,in_port="s5-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:a2 actions=output:"s5-eth2"
cookie=0x0, duration=64.511s, table=0, n_packets=2670, n_bytes=4037040, priority=1,ip,in_port="s5-eth2",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01 actions=output:"s5-eth1"
cookie=0x0, duration=43.296s, table=0, n_packets=2, n_bytes=84, priority=1,arp,in_port="s5-eth2",dl_src=00:00:00:00:00:a2,dl_dst=00:00:00:00:00:01 actions=output:"s5-eth1"
cookie=0x0, duration=35.002s, table=0, n_packets=1, n_bytes=42, priority=1,arp,in_port="s5-eth1",dl_src=00:00:00:00:00:a1,dl_dst=00:00:00:00:00:02 actions=output:"s5-eth2"
cookie=0x0, duration=12.073s, table=0, n_packets=213, n_bytes=322056, hard_timeout=15, priority=100,ip,in_port="s5-eth2",dl_src=00:00:00:00:00:a2,dl_dst=00:00:00:00:00:01 actions=drop
cookie=0x0, duration=0.009s, table=0, n_packets=11, n_bytes=16632, hard_timeout=15, priority=100,ip,in_port="s5-eth1",dl_src=00:00:00:00:00:a1,dl_dst=00:00:00:00:00:02 actions=drop

```

Figure 9 Regole installate in s5

Notiamo le due regole con priorità 100 caratterizzate da:

- sorgente 00:00:00:00:00:a1 (a1) e destinazione 00:00:00:00:00:02 (h2)
- sorgente 00:00:00:00:00:a2 (a2) e destinazione 00:00:00:00:00:01 (h1)

3.4. Simulazione Threshold dinamica

Con questa simulazione si vuole mostrare il comportamento della feature di threshold adattiva implementata tramite tecnica di moving average.

Per fare ciò è stato impostato un valore di ADAPTIVE_MULTIPLIER pari a 1.7.

Quindi un flusso, il cui valore di pps eccede la threshold (pari alla media degli ultimi 12 pacchetti moltiplicata per il moltiplicatore) viene bloccato.

In questo scenario è presente un flusso lecito da h1 a h2 con banda pari a 1 Mbps e un flusso da a1 a h2 con banda crescente. Nello specifico, il flusso generato da a1 aumenta secondo i seguenti step, ciascuno della durata di 10 s:

3 Mbps, 3.5 Mbps, 4 Mbps, 4.5 Mbps, 5 Mbps, 9 Mbps, 9 Mbps, 9 Mbps.

Simuliamo quindi un traffico intenso ma lecito fino a 5 Mbps, il successivo incremento repentino a 9Mbps segna l'inizio di un possibile traffico malevolo, come un attacco DoS.

Sono stati utilizzati i seguenti comandi:

- h1 iperf -c 10.0.0.2 -b 1M -p 5002 -t 80
- a1 ./ramp.sh

Si riporta lo script ramp.sh utilizzato da a1:

```
#!/bin/bash
DEST_IP="10.0.0.2"
PORT=5001
DURATA=10 # Secondi per ogni step

STEP_DI_BANDA=("3M" "3.5M" "4M" "4.5M" "5M" "9M" "9M" "9M")

for BANDA in "${STEP_DI_BANDA[@]}"
do
    echo "--- Test corrente: $BANDA ---"
    iperf -c $DEST_IP -u -b $BANDA -p $PORT -t $DURATA
    sleep 1
done
echo "Test completato."
```

Si riportano i risultati:

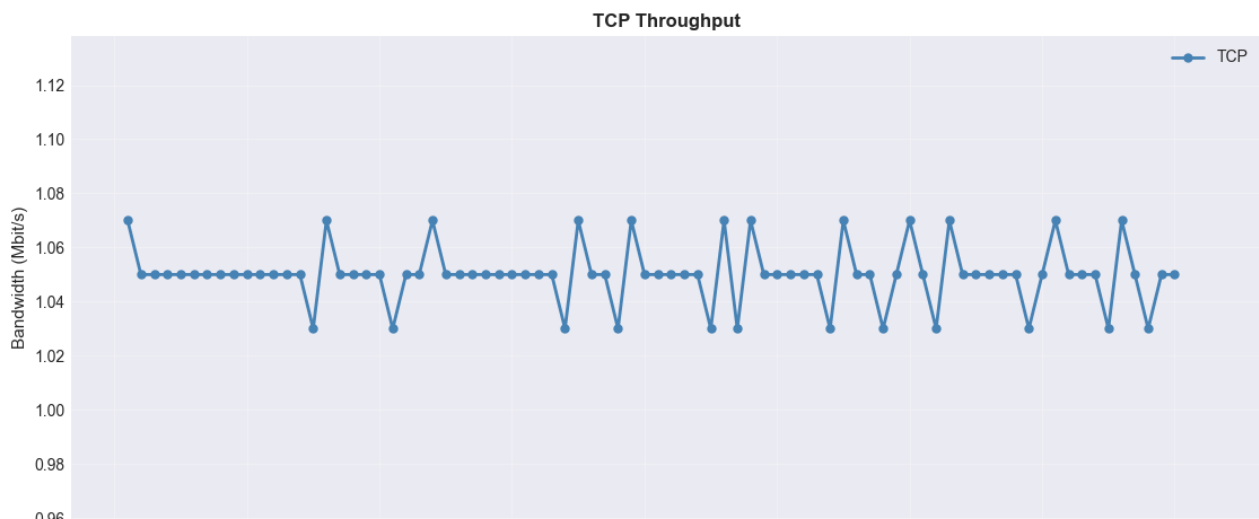


Figura 1 Flusso TCP da h1 a h2

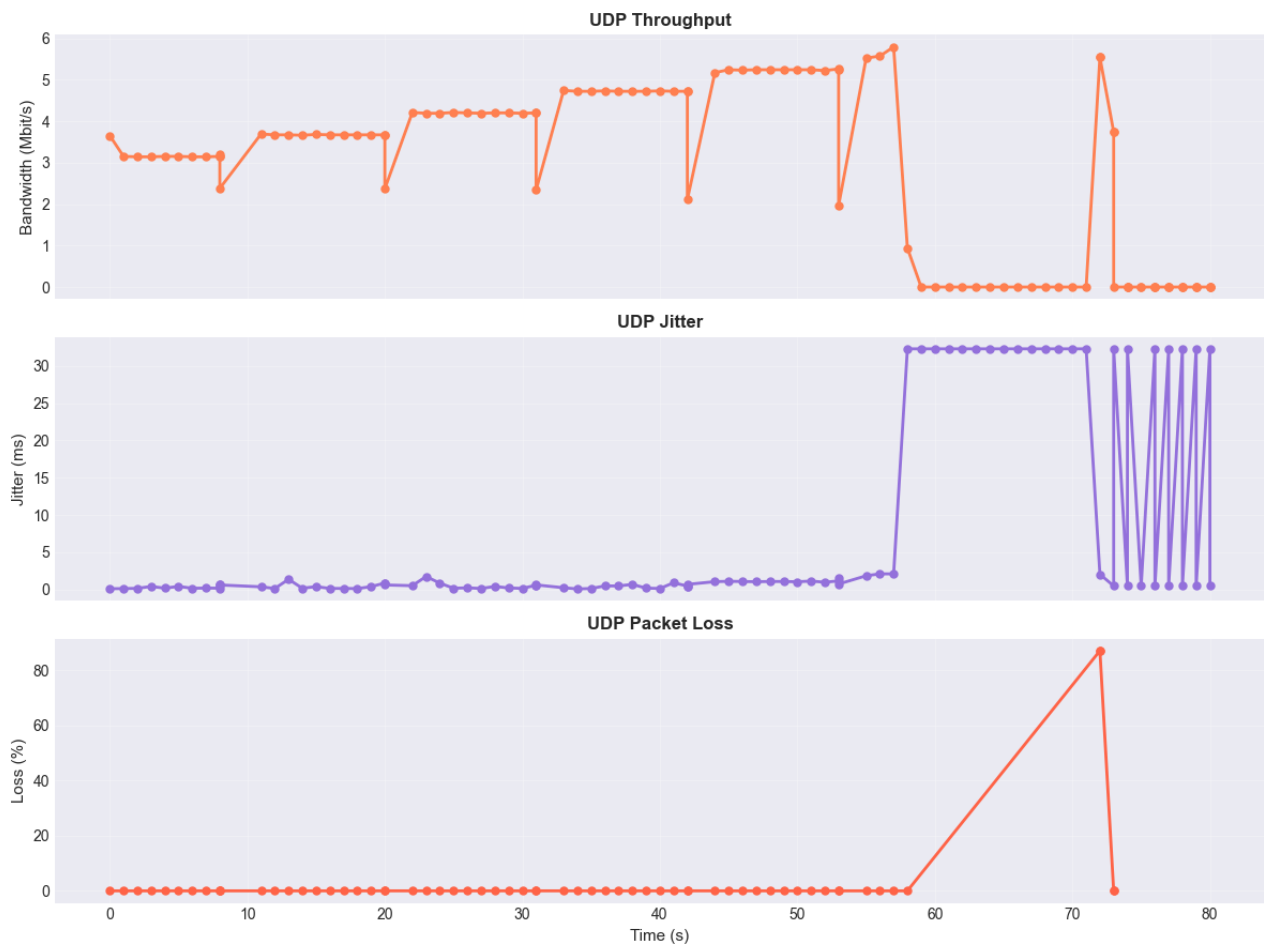


Figura 2 Flusso UDP da a1 a h2

Dai risultati si osserva che il flusso cresce progressivamente e viene bloccato solamente quando l'incremento di banda diventa troppo repentino, ovvero nel passaggio da 5 Mbps a 9 Mbps.

Notiamo dal prossimo screen del terminale del controller come la threshold si sia adattata dinamicamente al flusso, raggiungendo un valore di 760.97 e come il flusso venga bloccato durante l'ultimo incremento tra 5 Mbps e 9 Mbps (pps=803.87).

```
PolicyModule DECISION: Block flow (9, 1, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048) (pps=803.87 > adaptive_threshold=760.97)
EnforcementModule: BLOCKED flow (9, 1, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048) for 15s (pps=803.87) dpid=9 in_port=1 src=00:00:00:00:00:a1 dst=00:00:00:00:00:02 eth_type=2048
```

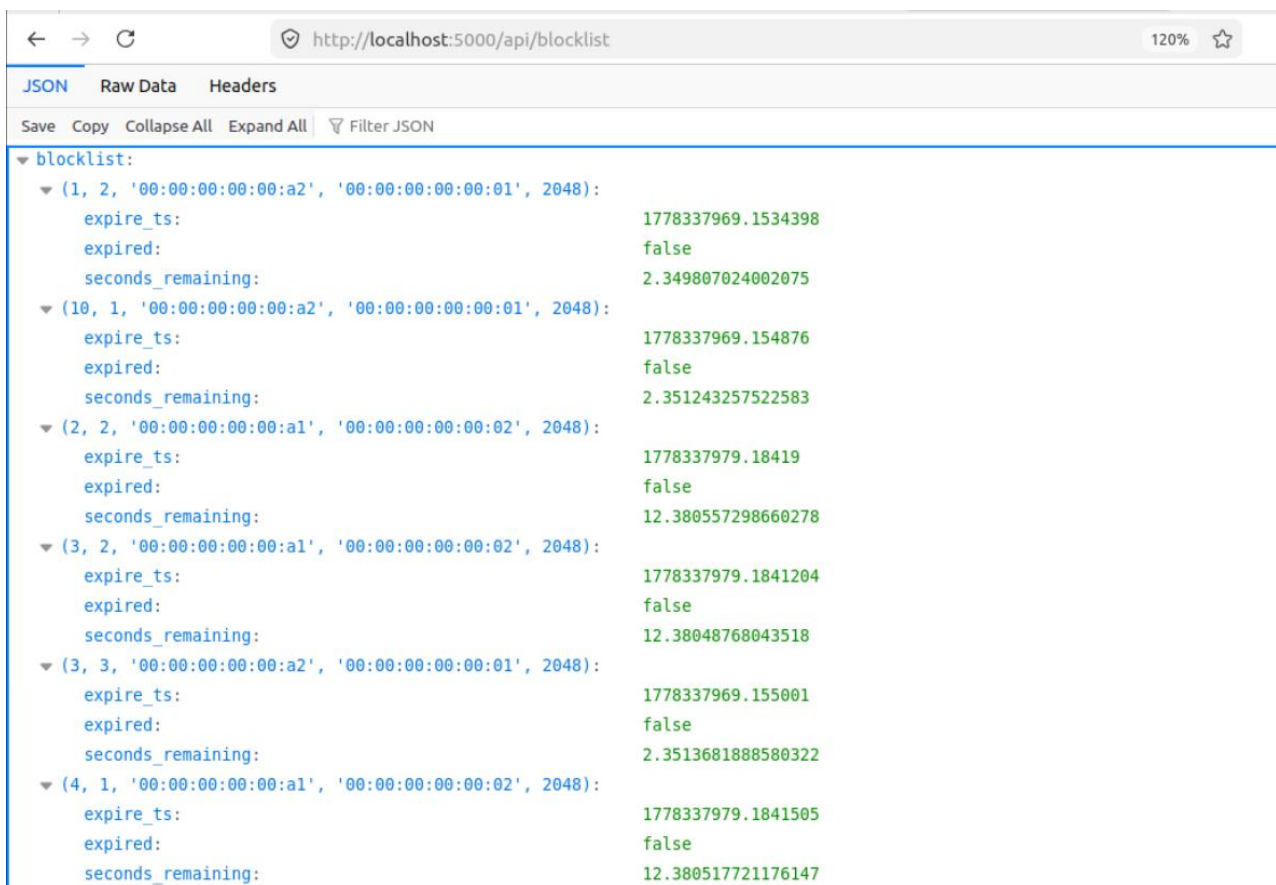

4. Utilizzo API REST

Al fine di mostrare l'utilizzo delle API REST, che permettono ad un admin esterno di gestire la blocklist del controller, simuliamo nuovamente lo scenario di attacco DoS doppio descritto precedentemente.

4.1. Metodo GET Blocklist

Il metodo GET per visualizzare l'intera blocklist.

Per ogni flusso nella blocklist, caratterizzato dalla sua *flow key*, viene riportato l'expire time in secondi, se il flusso è già expired e i secondi di blocco rimanenti.



The screenshot shows a web browser window with the address bar displaying `http://localhost:5000/api/blocklist`. The browser's developer tools are open, showing the JSON response of the GET request. The response is a list of 7 objects, each representing a flow in the blocklist. Each object contains three fields: `expire_ts`, `expired`, and `seconds_remaining`.

```
blocklist:
  (1, 2, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048):
    expire_ts: 1778337969.1534398
    expired: false
    seconds_remaining: 2.349807024002075
  (10, 1, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048):
    expire_ts: 1778337969.154876
    expired: false
    seconds_remaining: 2.351243257522583
  (2, 2, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048):
    expire_ts: 1778337979.18419
    expired: false
    seconds_remaining: 12.380557298660278
  (3, 2, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048):
    expire_ts: 1778337979.1841204
    expired: false
    seconds_remaining: 12.38048768043518
  (3, 3, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048):
    expire_ts: 1778337969.155001
    expired: false
    seconds_remaining: 2.3513681888580322
  (4, 1, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048):
    expire_ts: 1778337979.1841505
    expired: false
    seconds_remaining: 12.380517721176147
```

4.2. Metodo POST

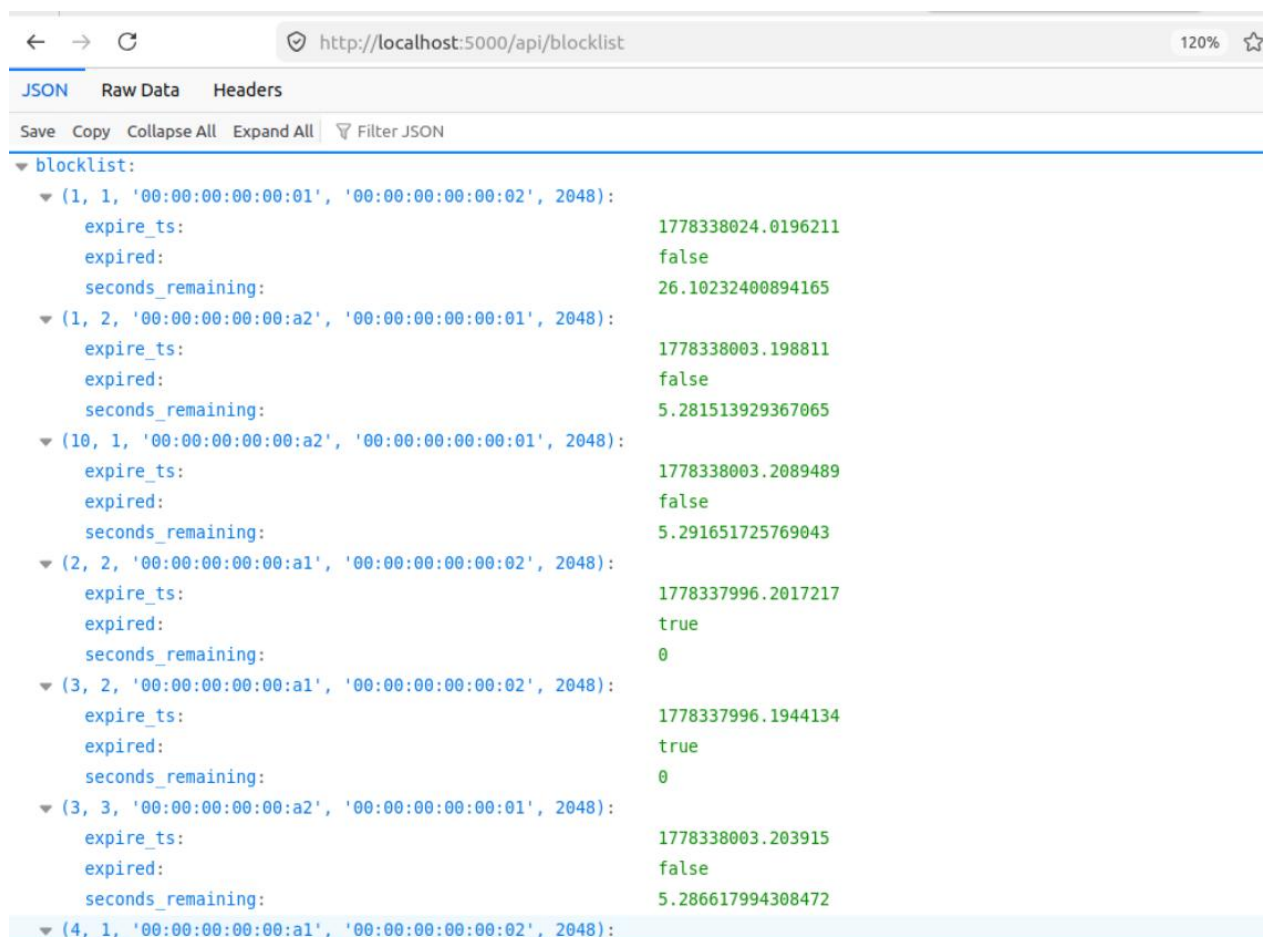
Il metodo POST permette ad un admin esterno di aggiungere una nuova regola di drop flow.

```
mirko@Ubuntu-NCIS:~/shared_folder/Share Folder/6) progetto finale/test/test controller/Doppio D05$ curl -i -X POST "
$INDIRIZZO/api/blocklist" \
-H "Content-Type: application/json" \
-d '{
  "flow_key": [1, 1, "00:00:00:00:00:01", "00:00:00:00:00:02", 2048],
  "duration": 30
}'
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.8 Python/3.10.12
Date: Sat, 09 May 2026 14:44:01 GMT
Content-Type: application/json
Content-Length: 180
Connection: close

{"block_duration_sec":30,"expire_ts":1778337871.4843018,"flow_key": "(1, 1, '00:00:00:00:00:01', '00:00:00:00:00:02',
2048)", "message": "Flow added to blocklist", "status": "success"}
```

In questo caso stiamo andando a bloccare il flusso lecito tra h1 e h2 con un blocco di durata 30 secondi.

Chiamando nuovamente il metodo GET Blocklist, notiamo che la regola è stata installata correttamente.



The screenshot shows a web browser window with the address bar displaying `http://localhost:5000/api/blocklist`. The page content is a JSON array of 8 objects, each representing a flow rule. The objects are expanded to show their properties: `expire_ts`, `expired`, and `seconds_remaining`. The first three rules have `expired` set to `false` and a non-zero `seconds_remaining`. The fourth and fifth rules have `expired` set to `true` and `seconds_remaining` set to `0`. The last two rules have `expired` set to `false` and a non-zero `seconds_remaining`.

Flow Key	expire_ts	expired	seconds_remaining
(1, 1, '00:00:00:00:00:01', '00:00:00:00:00:02', 2048)	1778338024.0196211	false	26.10232400894165
(1, 2, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048)	1778338003.198811	false	5.281513929367065
(10, 1, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048)	1778338003.2089489	false	5.291651725769043
(2, 2, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048)	1778337996.2017217	true	0
(3, 2, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048)	1778337996.1944134	true	0
(3, 3, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048)	1778338003.203915	false	5.286617994308472
(4, 1, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048)			

4.3. Metodo DELETE

Il metodo DELETE permette di eliminare una regola precedentemente installata.

```
mirko@Ubuntu-NCIs:~/shared_folder/Share Folder/6) progetto finale/test/Test controller/Doppio D05$ FLOW_KEY="(1, 1, '00:00:00:00:00:01', '00:00:00:00:00:02', 2048)"
mirko@Ubuntu-NCIs:~/shared_folder/Share Folder/6) progetto finale/test/Test controller/Doppio D05$ ENCODED=$(python3 -c "import urllib.parse; print(urllib.parse.quote(\"$FLOW_KEY\", safe=''))")
curl -i -X DELETE "$INDIRIZZO/api/blocklist/$ENCODED"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.10.12
Date: Sat, 09 May 2026 14:45:43 GMT
Content-Type: application/json
Content-Length: 114
Connection: close

{"flow_key": "(1, 1, '00:00:00:00:00:01', '00:00:00:00:00:02', 2048)", "message": "Flow removed", "status": "success"}
```

Richiamando il metodo GET Blocklist notiamo come il flusso è stato correttamente rimosso dalla Blocklist.

```
▼ blocklist:
  ▼ (1, 2, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048):
    expire_ts: 1778337747.8292441
    expired: true
    seconds_remaining: 0
  ▼ (10, 1, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048):
    expire_ts: 1778337798.9166622
    expired: false
    seconds_remaining: 11.931315898895264
  ▼ (2, 2, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048):
    expire_ts: 1778337791.9096143
    expired: false
    seconds_remaining: 4.924268007278442
  ▼ (3, 2, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048):
    expire_ts: 1778337791.909461
    expired: false
    seconds_remaining: 4.92411470413208
  ▼ (3, 3, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048):
    expire_ts: 1778337798.9126828
    expired: false
    seconds_remaining: 11.92733645439148
  ▼ (4, 1, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048):
    expire_ts: 1778337791.9094172
    expired: false
    seconds_remaining: 4.924070835113525
  ▼ (4, 3, '00:00:00:00:00:a2', '00:00:00:00:00:01', 2048):
    expire_ts: 1778337798.9166865
    expired: false
    seconds_remaining: 11.931340217590332
  ▼ (5, 1, '00:00:00:00:00:a1', '00:00:00:00:00:02', 2048):
    expire_ts: 1778337791.9095182
    expired: false
```

4.4. Metodo GET Info

Il metodo get info permette ad un admin esterno di ricevere informazioni generiche sul controller come:

- Flussi attualmente bloccati
- Tempo di blocco impostato di default
- Flussi expired ma ancora in blocklist
- Totale flussi bloccati

